COMPTE RENDU D'ATELIER

Développement de MediaTekFormation

Contexte

Cet atelier s'inscrit dans le cadre du développement de MediaTekFormation pour l'entreprise MédiaTek86, permettant à leurs clients d'accéder à des ressources, formations, etc.

Technologies utilisées

PHP 8 / Symfony 6
Twig (moteur de templates)
Tailwind CSS
JavaScript / jQuery / DataTables
PHPUnit (tests unitaires et d'intégration)
Doctrine ORM
MySQL / PlanetHoster
Git / GitHub Actions (CI/CD)
SonarLint

Sommaire

Mission 1 : Nettoyer le code existant et ajouter une fonctionnalité
- Tâche 1 : Correction des erreurs SonarLint
- Tâche 2 : Affichage du nombre de formations
- Tâche 3 : Ajout des boutons de tri

Mission 2 : Coder la partie back office
- Tâche 1 : Gérer les formations
- Tâche 2 : Gérer les playlists
- Tâche 3 : Gérer les catégories
- Mise en place technique du back office
- Tâche 4 : Ajouter l'authentification

Mission 3 : Tester et documenter
- Tâche 1 : Réaliser les tests
- Tâche 2 : Créer la documentation technique
- Tâche 3 : Créer la documentation utilisateur

Mission 4 : Déployer le site et gérer le déploiement continu
- Tâche 1 : Déployer le site

Bilan final
- Objectifs atteints
- Problèmes rencontrés

Mission 1 : Nettoyer le code existant et ajouter une fonctionnalité

Tâche 1 : Correction des erreurs SonarLint

J'ai corrigé les erreurs détectées par SonarLint dans les dossiers src, template et public. Pour ce faire, j'ai effectué un clic-droit sur le dossier concerné, puis sélectionné Tools → Analyze with SonarLint.

En fonction des erreurs, j'ai cherché ou non des solutions sur internet. Certaines erreurs étaient évidentes (par exemple, supprimer un espace en fin de ligne), tandis que d'autres s'avéraient plus complexes. Par exemple, l'erreur S105 pour laquelle je n'ai trouvé aucune documentation et que je n'ai pas pu résoudre.

Tâche 2 : Affichage du nombre de formations

Pour l'affichage du nombre de formations, j'ai opté pour la solution la plus propre possible, en l'implémentant en une seule ligne de code (3 si l'on compte les accolades).

```
<td class="text-center">
    {{ playlists[k].formations|length }}
</td>
```

Tâche 3 : Ajout des boutons de tri par nombre de formations

Pour l'ajout des deux boutons de tri, j'ai dans un premier temps copié-collé les boutons existants de la section Playlist et les ai placés dans une rubrique Formation.

```
<th class="text-left align-top" scope="col">
    Formations<br />
    <a href="{{ path('playlists.sort', {champ:'name', ordre:'ASC'}) }}" class="btn btn-info btn-sm active" role="button" aria-pressed="true"><</a>
    <a href="{{ path('playlists.sort', {champ:'name', ordre:'DESC'}) }}" class="btn btn-info btn-sm active" role="button" aria-pressed="true">></a>
</th>
```

Je me suis servi de playlist.sort pour les deux boutons, en changeant simplement le champ name par nbformations, qui correspond le mieux à la logique métier.

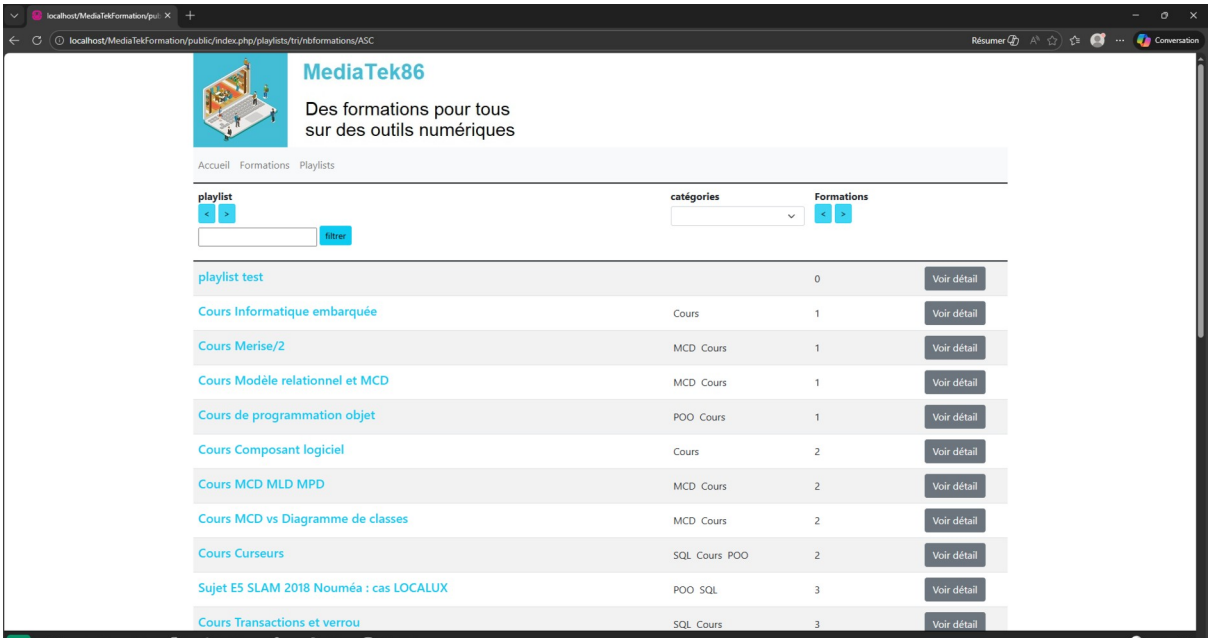
```
{champ:'nbformations', ordre:'ASC'})
{champ:'nbformations', ordre:'DESC'})
```

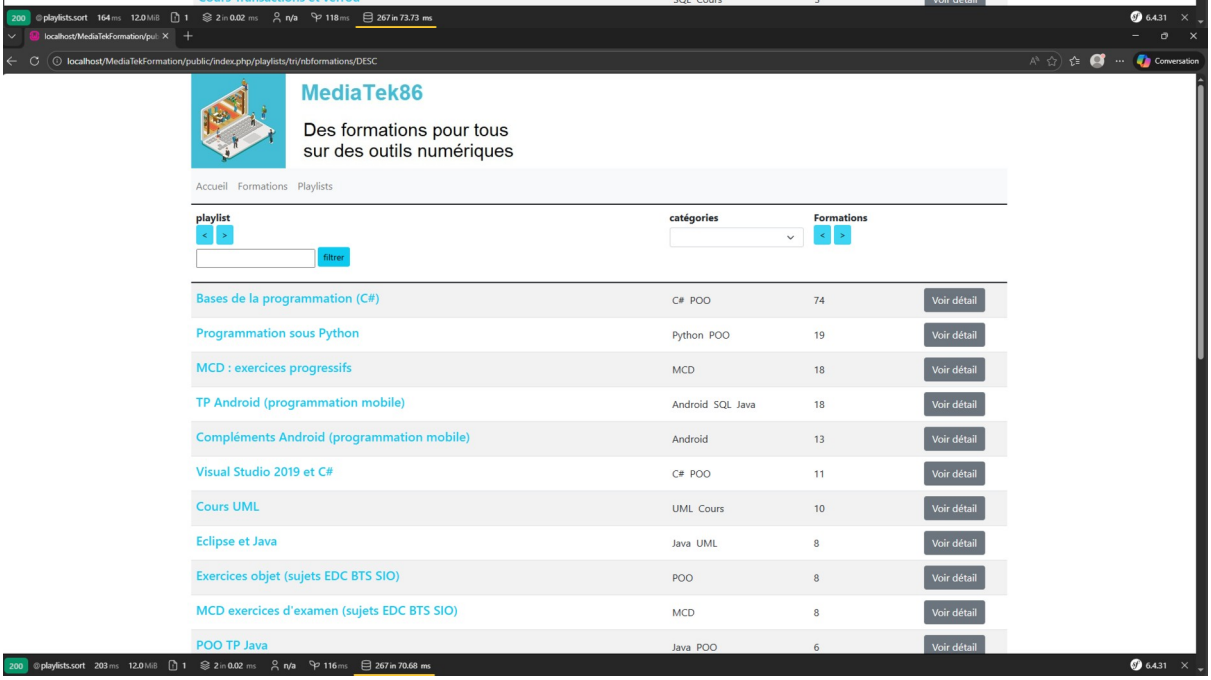
Il m'a ensuite fallu modifier la méthode sort dans PlaylistController.php pour qu'elle puisse gérer à la fois le champ name et le champ nbformation. Pour ce faire, j'ai remplacé le if par un switch, ajouté le cas nbformation et la future fonction associée.

```
#[Route('/playlists/tri/{champ}/{ordre}', name: 'playlists.sort')]
public function sort($champ, $ordre): Response{
    switch($champ){
        case "name":
            $playlists = $this->playlistRepository->findAllOrderByName($ordre);
            break;
        case "nbformations":
            $playlists = $this->playlistRepository->findAllOrderByNbFormations($ordre);
            break;
        default:
            // fallback
            $playlists = $this->playlistRepository->findAllOrderByName('ASC');
            break;
    }
    $categories = $this->categorieRepository->findAll();
    return $this->render("pages/playlists.html.twig", [
        'playlists' => $playlists,
        'categories' => $categories
    ]);
}
```

J'ai ensuite créé la fonction findAllOrderByNbFormation dans PlaylistRepository.php, en partant de la fonction findAllOrderByName existante, en ajoutant un comptage du nombre de formations et en changeant le orderBy p.name par nbformations.

```php
public function findAllOrderByNbFormations(string $ordre): array
{
    return $this->createQueryBuilder('p')
        ->leftJoin('p.formations', 'f')
        ->addSelect('COUNT(f) AS HIDDEN nbformations')
        ->groupBy('p.id')
        ->orderBy('nbformations', $ordre)
        ->getQuery()
        ->getResult();
}
```





Mission 2 : Coder la partie back office

L'objectif de cette mission est de permettre l'ajout, la modification et la suppression de formations, catégories et playlists, de manière sécurisée via une interface d'administration.

Tâche 1 : Gérer les formations (5h)

J'ai créé les pages et contrôleurs nécessaires pour permettre l'ajout, la modification et la suppression d'une formation depuis le back office.

Tâche 2 : Gérer les playlists (5h)

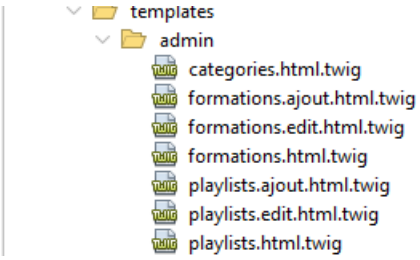
De la même manière que pour les formations, j'ai mis en place les fonctionnalités CRUD pour les playlists.

Tâche 3 : Gérer les catégories (3h)

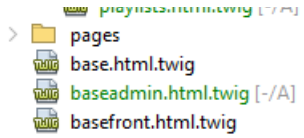
Pour les catégories, j'ai implémenté l'ajout et la suppression (sans modification, car une catégorie est identifiée uniquement par son nom).

Mise en place technique du back office

J'ai décidé de traiter ces trois fonctionnalités en parallèle car elles suivent la même logique. Dans un premier temps, j'ai créé un dossier admin dans templates et y ai placé mes fichiers Twig dédiés à l'administration.



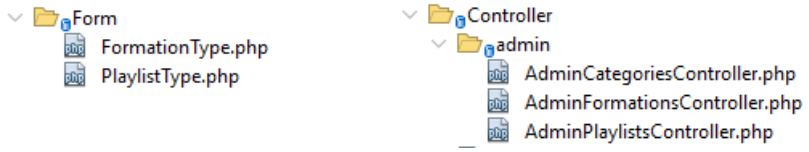
J'ai ensuite créé un fichier baseadmin.html.twig, servant de layout de base pour toutes les pages admin, au même titre que basefront.html.twig et base.html.twig sont utilisés pour le reste du site.



Une fois les fichiers Twig complétés pour le front, j'ai créé les formulaires Symfony dans le dossier src/Form : FormationType.php et PlaylistType.php.

```
    <h2 class="mt-3">Catégories</h2>

    <form method="POST" action="{{ path('admin.categories.ajout') }}" class="form-inline mt-3 mb-3">
        <div class="form-group mr-2">
            <label for="name" class="mr-2">Nouvelle catégorie :</label>
            <input type="text" id="name" name="name" class="form-control" required>
        </div>
        <button type="submit" class="btn btn-primary btn-sm">Ajouter</button>
    </form>

    <table class="table table-striped">
        <thead>
            <tr>
                <th>Nom de la catégorie</th>
                <th>Nb formations</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
        {% for categorie in categories %}
            <tr>
                <td>{{ categorie.name }}</td>
                <td>{{ categorie.formations|length }}</td>
                <td>
                    <a href="{{ path('admin.categories.suppr', {id: categorie.id}) }}"
                       class="btn btn-danger btn-sm"
                       onclick="return confirm('Supprimer la catégorie {{ categorie.name }} ?');">
                        Supprimer
                    </a>
                </td>
            </tr>
        {% else %}
            <tr><td colspan="3">Aucune catégorie trouvée.</td></tr>
        {% endfor %}
        </tbody>
    </table>

{% endblock %}
```

Pour les contrôleurs, j'ai créé un dossier admin dans src/Controller et y ai ajouté un contrôleur par entité. Ces contrôleurs sont inspirés du projet mesvoyages et adaptés à MediaTekFormation.



Tâche 4 : Ajouter l'authentification (4h)

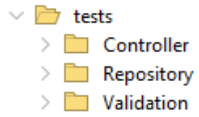
Afin de sécuriser l'accès au back office, j'ai ajouté une page de connexion. J'ai réutilisé le code du projet mesvoyages concernant l'authentification, en l'adaptant au projet.

Les fichiers repris et adaptés sont les suivants : UserRepository.php, User.php (l'entité), LoginController.php, le template templates/login/index.html.twig, ainsi que UserFixture.php dans le dossier DataFixtures pour initialiser les utilisateurs.

Mission 3 : Tester et documenter

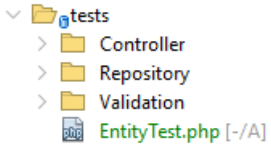
Tâche 1 : Réaliser les tests (7h)

Pour les tests, j'ai appliqué la même approche que pour le projet mesvoyages, en l'adaptant à MediaTekFormation. J'ai créé 3 dossiers dans le dossier test du projet.



Tests unitaires

J'ai créé une classe EntityTest directement dans le dossier test, avec le namespace App\Tests,



héritant de TestCase du framework PHPUnit. J'ai créé une fonction de test par entité, en commençant par testgetPublishedAtString(),

```php
namespace App\Tests;

use PHPUnit\Framework\TestCase;
use App\Entity\Formation;
/**
 * Description of EntityTest
 *
 * @author SlameX
 */
class EntityTest extends TestCase {

    public function testgetPublishedAtString(){
        $Formation = new Formation();
        $Formation->setPublishedAt(new \DateTime("2005-11-18"));
        $this->assertEquals("18/11/2005", $Formation->getPublishedAtString());
    }
}
```

puis j'ai lancé les tests via une commande CMD.

```
c:\wamp64\www\MediaTekFormation>php bin/phpunit
PHPUnit 9.6.31 by Sebastian Bergmann and contributors.

Testing
.                                                                   1 / 1 (100%)

Time: 00:00.019, Memory: 6.00 MB

OK (1 test, 1 assertion)
```

EntityTest.php [-/A]

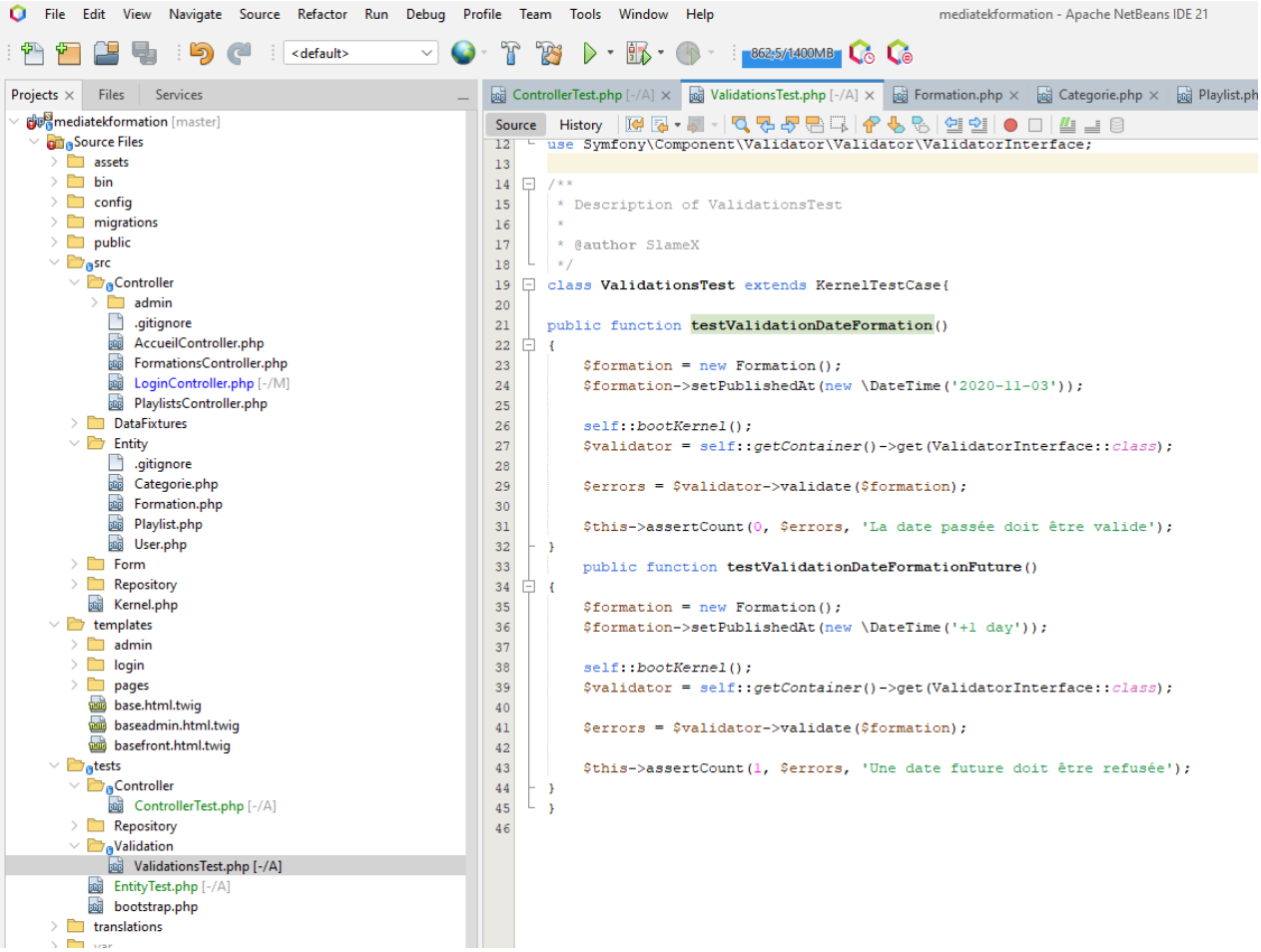
```php
use App\Entity\Formation;
use App\Entity\Categorie;
use App\Entity\Playlist;
/**
 * Description of EntityTest
 *
 * @author SlameX
 */
class EntityTest extends TestCase {

    public function testgetPublishedAtString(){
        $Formation = new Formation();
        $Formation->setPublishedAt(new \DateTime("2005-11-18"));
        $this->assertEquals("18/11/2005", $Formation->getPublishedAtString());
    }

    public function testgetNameCategorie()
    {
        $Categorie = new Categorie();
        $Categorie->setName('Test Name');

        $this->assertSame('Test Name', $Categorie->getName());
    }

    public function testgetNamePlaylist(){
        $Playlist = new Playlist();
        $Playlist->setName('Test Name');

        $this->assertSame('Test Name', $Playlist->getName());
    }
}
```

Administrateur : Invite de commandes

```
c:\wamp64\www\MediaTekFormation>php bin/phpunit
PHPUnit 9.6.31 by Sebastian Bergmann and contributors.

Testing
...                                                                 3 / 3 (100%)

Time: 00:00.021, Memory: 8.00 MB

OK (3 tests, 3 assertions)

c:\wamp64\www\MediaTekFormation>
```

Tests d'intégration - Validations

J'ai créé la classe ValidationsTest avec le namespace App\Tests\Validations, afin de tester les règles de validation des entités.



Tests d'intégration - Repository

Les tests sur les Repository ont été réalisés pour s'assurer du bon fonctionnement des requêtes en base de données.

Tests fonctionnels

Des tests fonctionnels ont été réalisés pour vérifier l'accès à l'accueil, les tris et les filtres de l'application.

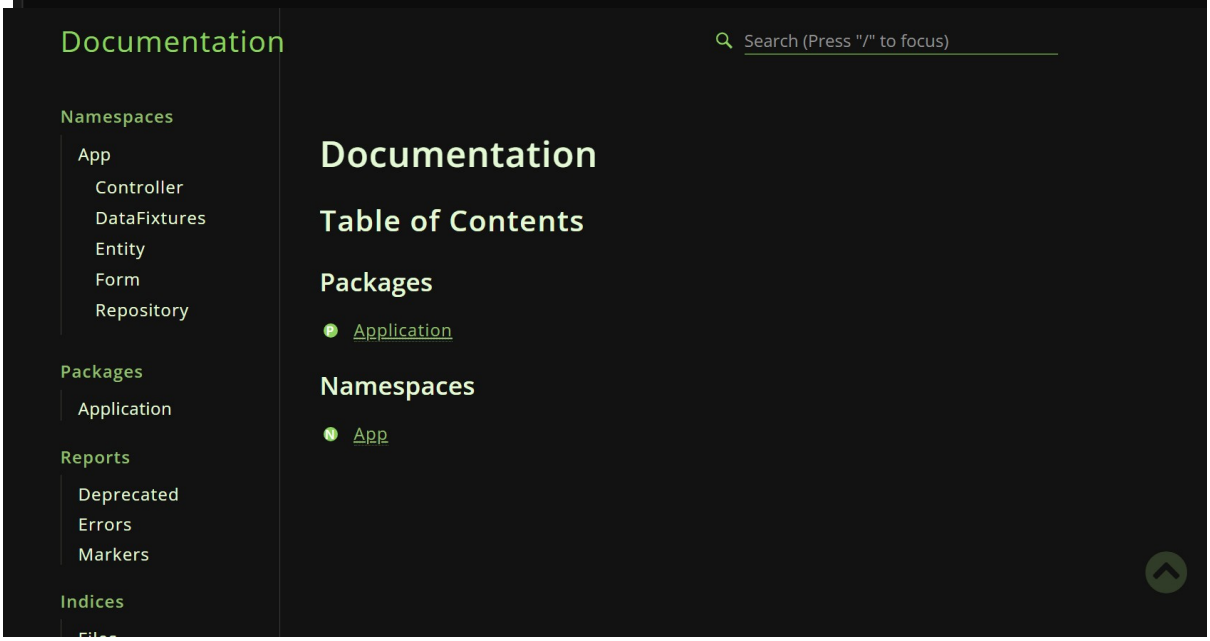
Tests de compatibilité navigateurs

Des tests de compatibilité ont été effectués sur Chrome et Firefox pour s'assurer du bon rendu et du bon fonctionnement de l'application.

Tâche 2 : Créer la documentation technique (1h)

J'ai généré la documentation technique pour l'ensemble de l'application, à l'exception du code généré automatiquement par Symfony.

```
c:\wamp64\bin\php\php8.4.0>php phpDocumentor.phar ^
Plus ? -d C:\wamp64\www\MediaTekFormation\src ^
Plus ? -t C:\wamp64\www\MediaTekFormation\docs ^
Plus ? --ignore vendor/*,tests/*,var/*,node_modules/*
Failed loading E:/wamp64/bin/php/php8.4.0/zend_ext/php_xdebug-3.4.0beta1-8.4-x86_64.dll
phpDocumentor 3.9.1

Parsing source files
 19/19 [============================] 100%
Applying transformations (can take a while)
 22/22 [============================] 100%
All done in 27 seconds!

c:\wamp64\bin\php\php8.4.0>
```



Tâche 3 : Créer la documentation utilisateur (2h)

J'ai réalisé une vidéo de 5 minutes présentant l'ensemble des fonctionnalités de l'application, à destination des utilisateurs finaux.

Mission 4 : Déployer le site et gérer le déploiement continu

Tâche 1 : Déployer le site (2h)

J'ai tenté de déployer le site chez l'hébergeur PlanetHoster. J'ai pu configurer et gérer la base de données sur cet hébergeur, mais je n'ai pas réussi à envoyer les fichiers du projet, que ce soit via FTP ou en les compressant en archive ZIP.

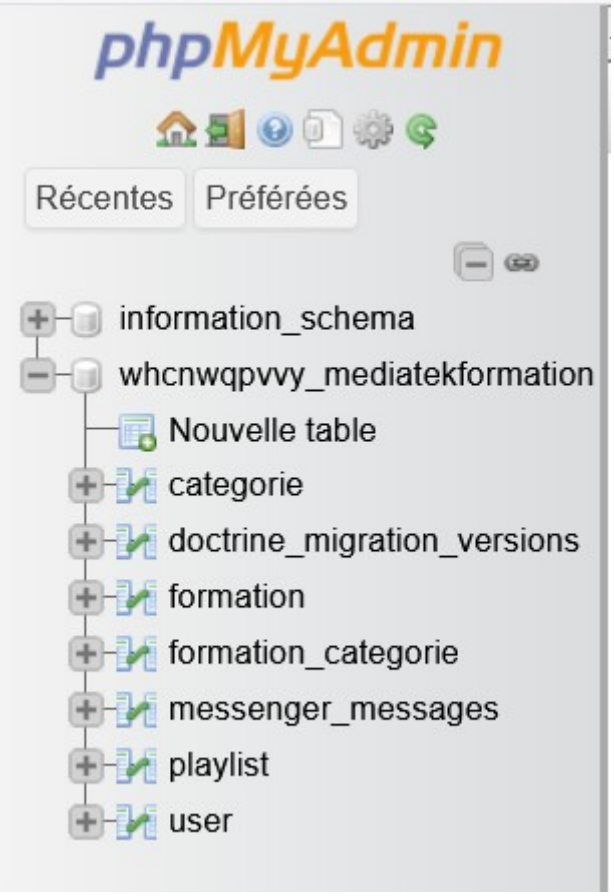


	Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐	categorie	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐	doctrine_migration_versions	Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb3_unicode_ci	16,0 kio	-
☐	formation	Parcourir Structure Rechercher Insérer Vider Supprimer	238	InnoDB	utf8mb4_unicode_ci	160,0 kio	-
☐	formation_categorie	Parcourir Structure Rechercher Insérer Vider Supprimer	278	InnoDB	utf8mb4_unicode_ci	48,0 kio	-
☐	messenger_messages	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐	playlist	Parcourir Structure Rechercher Insérer Vider Supprimer	28	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐	user	Parcourir Structure Rechercher Insérer Vider Supprimer	1	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
	7 tables	**Somme**	**561**	**MyISAM**	**latin1_swedish_ci**	**320,0 kio**	**0 o**

Tout cocher Avec la sélection :

Bilan final

Objectifs atteints

- Correction des erreurs de code détectées par SonarLint
- Affichage du nombre de formations et ajout des boutons de tri
- Développement complet du back office (formations, playlists, catégories)
- Mise en place de l'authentification sécurisée pour le back office
- Réalisation des tests unitaires, d'intégration et fonctionnels avec PHPUnit
- Génération de la documentation technique et création de la vidéo utilisateur
- Configuration de la base de données sur PlanetHoster
- Mise en place du déploiement continu via GitHub Actions

Problèmes rencontrés

- Erreur SonarLint S105 : aucune documentation disponible, non résolue
- Impossibilité d'envoyer les fichiers du projet sur PlanetHoster (FTP et ZIP)
- Difficultés liées aux permissions IIS pour l'accès aux fichiers statiques depuis PHP
- Configuration du serveur SMTP pour l'envoi de mails depuis Symfony